

[PSI3471 aula 14 início]

Nota 06/05/2025: A versão atual do tensorflow é 2.18 e as observações abaixo podem não valer.  
Nota: Problema no Tensorflow 2.17.0

(16/08/2024): Estranhamente, estou obtendo resultados bastante piores executando o programa 5 desta apostila usando Colab com tensorflow v2.17.0 do que aqueles que tinha obtido anos atrás (tabela T1). Executando programa 5 no computador local com Tensorflow v2.13.1, obtive resultado praticamente idêntico ao que tinha obtido anos atrás.

(19/08/2024): Consegui instalar tensorflow v2.15.0 no Colab. Fazendo isso, obtive o mesmo resultado da execução antiga (Tabela T1). A receita para instalar tensorflow 2.15.0 é a seguinte:

|             |                                                                                       |
|-------------|---------------------------------------------------------------------------------------|
| Instalação: | <pre>!pip uninstall tensorflow -y --quiet !pip install tensorflow==2.15 --quiet</pre> |
|-------------|---------------------------------------------------------------------------------------|

Depois, pode precisar dar “restart runtime” (reiniciar sessão). Execute os seguintes comandos se quiser verificar se teve sucesso na instalação.

|              |                                                                                                                        |
|--------------|------------------------------------------------------------------------------------------------------------------------|
| Verificação: | <pre>import tensorflow as tf; print("Versao tf:",tf.__version__) import sys; print("Versao python:",sys.version)</pre> |
| Resposta:    | <pre>Versao tf: 2.15.0 Versao python: 3.10.12 (main, Jul 29 2024, 16:56:48) [GCC 11.4.0]</pre>                         |

Tabela T1: Métricas de desempenho na execução antiga, e nas execuções com Tensorflow versões 2.17, 2.13 e 2.15. Os resultados do Tensorflow 2.17 são muito piores os que de outras versões.

|                                        | Resultado antigo     | Google Colab<br>Tensorflow v2.17.0<br>com bug | Computador local<br>Tensorflow v2.13.1<br>2024 | Google Colab<br>Tensorflow v 2.15.0<br>2024 |
|----------------------------------------|----------------------|-----------------------------------------------|------------------------------------------------|---------------------------------------------|
| Após 20 épocas de<br>transfer learning | Test accuracy 90.00% | Test accuracy 85.10%<br>Test loss 0.4240      | Test accuracy 89.82%<br>Test loss 0.3096       | Test accuracy 90.05%<br>Test loss 0.2999    |
| Após 10 épocas de<br>fine tuning       | Test accuracy 96.37% | Test accuracy 94.41%<br>Test loss 0.1654      | Test accuracy 96.46%<br>Test loss 0.1121       | Test accuracy 96.38%<br>Test loss 0.1133    |

# Transfer Learning

Para treinar uma rede neural na prática usando um certo conjunto de dados customizado (por exemplo, para classificar uma imagem de raio-X como câncer ou não-câncer) não se costuma iniciar o treino “do zero” com parâmetros iniciais aleatórios. O habitual é pegar uma rede já “pré-treinada” para outras tarefas e adaptá-la para o problema específico. Isto chama-se “transfer learning”. É o que veremos nesta aula.

Nota: Muitas vezes, os termos “transfer learning” e “fine tuning” são usados como sinônimos.

## I. Classificação de rostos em M/F

### 1. Rede “tipo LeNet” treinada “do zero”

O site abaixo contém vários conjuntos de imagens de faces humanas:

<http://fei.edu.br/~cet/facedatabase.html>

Desse site, peguei o conjunto com faces frontais de 200 pessoas (400 imagens coloridas com 360×260 pixels) de rostos frontais com expressão neutra (\*a.jpg) e sorridente (\*b.jpg), alinhadas manualmente. Dessas imagens, metade são rostos masculinos e metade são femininos. Recortei as bordas dessas imagens, para que fiquem com 280×200 pixels. As 400 imagens assim obtidas estão em:

<http://www.lps.usp.br/hae/apostila/feiCorCrop.zip> (compactado) OU

<https://drive.google.com/drive/folders/1WdR28ZopUOUG9Y7grJ7B0bvlezU3m-P3?usp=sharing> (descompactado)

Nesse ZIP, também há arquivo “todos.csv”, com lista de nomes de arquivos seguida de classificação (0=masculino e 1=feminino):

```
095a.jpg;0  
095b.jpg;0  
096a.jpg;1  
096b.jpg;1
```

Indicando que a figura 95 é de homem e 96 é de mulher (figura 1).

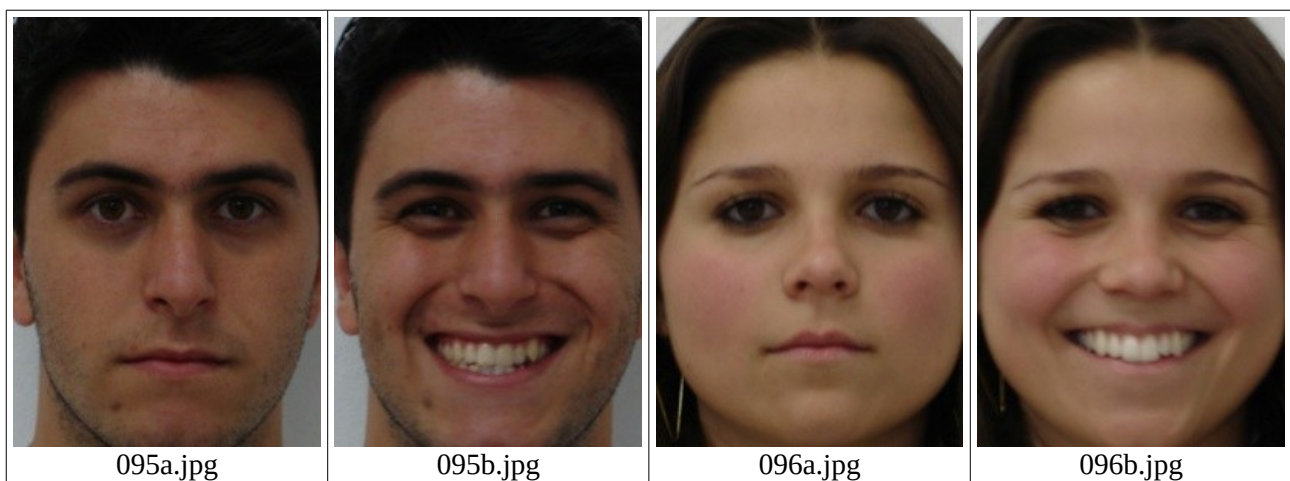


Figura 1: Algumas imagens do BD feiCorCrop.

A seguinte célula Colab baixa e descompacta esse conjunto no diretório local, se ainda não estiver lá.

```
1 url='http://www.lps.usp.br/hae/apostila/feiCorCrop.zip'
2 import os; nomeArq=os.path.split(url)[1]
3 if not os.path.exists(nomeArq):
4     print("Baixando o arquivo", nomeArq, "para diretorio default", os.getcwd())
5     os.system("wget -nc -U 'Firefox/50.0' "+url)
6 else:
7     print("O arquivo", nomeArq, "ja existe no diretorio default", os.getcwd())
8     print("Descompactando arquivos novos de", nomeArq)
9 os.system("unzip -u "+nomeArq)
```

Programa 1: “Célula de código Colab” para baixar e descompactar BD.

<https://colab.research.google.com/drive/10P97EWEih4XJrb-6gQYYM1IWw60KOLDQ#scrollTo=3CzP-xmHHqh5&uniqifier=1>

### ***k-fold Cross Validation***

Como a quantidade de dados de treino/teste é pequena (400 imagens), é difícil estimar com precisão a taxa de acerto. Não adianta repetir treino/teste várias vezes, pois pode ser que a divisão nos conjuntos treino/teste seja fácil/difícil de classificar, introduzindo viés na acuracidade calculada. Para superar esta dificuldade, vamos usar *k-fold cross validation*.

A figura abaixo mostra 5-fold cross validation ( $k=5$ ). Os dados são divididos em 5 subconjuntos. Em cada iteração, um subconjunto é usado para teste e os 4 subconjuntos restantes são usados para treino. É possível usar uma parte dos subconjuntos de treino para validação. Com isso, obtemos 5 medidas de acuracidade. Pode-se calcular a média dessas 5 acuracidades.

|            |        |        |        |        |        |
|------------|--------|--------|--------|--------|--------|
| Iteração 1 | Teste  | Treino | Treino | Treino | Treino |
| Iteração 2 | Treino | Teste  | Treino | Treino | Treino |
| Iteração 3 | Treino | Treino | Teste  | Treino | Treino |
| Iteração 4 | Treino | Treino | Treino | Teste  | Treino |
| Iteração 5 | Treino | Treino | Treino | Treino | Teste  |

Figura: 5-fold cross-validation.

*Nota:* Como veremos, obteremos acuracidade média de teste de 87,57% fazendo 5-fold cross validation. Nas versões antigas desta apostila, eu dividia o conjunto em treino, validação e teste fixos. Fazendo isso, estava obtendo acuracidades de 95,3% para validação e 94,8% para teste. Separando conjunto pequeno em treino-teste, há o perigo de criar conjunto de teste muito fácil ou muito difícil de classificar.

### ***Repetir 7 vezes 5-fold cross validation***

Como cada execução de 5-fold cross validation ainda não dá resultado de desempenho confiável, repeti 7 vezes o processo de 5-fold cross validation (fazendo cada vez diferente divisão em 5 subconjuntos), treinando e testando 35 vezes.

### Rede tipo LeNet

O programa 2 avalia o desempenho de rede “tipo LeNet” (sem data augmentation) e faz treino “do zero” usando parâmetros inicializados aleatoriamente. Para obter uma taxa de acerto confiável, repeti o programa 7 vezes, em cada repetição usando 5-fold cross validation. A acuracidade média de 35 testes é 87,57% (tabela 1).

*Exercício:* Acrescente data augmentation no programa 2 para verificar se é possível aumentar a taxa de acertos de teste/validação.

*Exercício:* Introduza outras alterações no programa 2 para aumentar a taxa de acertos de teste/validação.

*Exercício:* Utilize outros modelos de redes (como VGG, Inception e ResNet) e faça treino “do zero”, para verificar se é possível obter taxa de acerto substancialmente melhor.

Tabela 1: Acuracidades obtidas em 7 repetições de 5-fold cross validation (35 execuções ao todo, programa 2, usando Tensorflow 2.17).

|                                                 | repet #0 | repet #1 | repet #2 | repet #3 | repet #4 | repet #5 | repet #6 | Média  |
|-------------------------------------------------|----------|----------|----------|----------|----------|----------|----------|--------|
| Acuracidade de teste em 5-fold cross validation | 89.75%   | 87.75%   | 86.75%   | 88.50%   | 87.00%   | 87.50%   | 87.50%   | 87.57% |

[illegible]

Programa 2: Rede “tipo LeNet” sem data augmentation para classificar feiCorCrop em M/F. Chega-se à acuracidade de 87,57%. Execute primeiro o programa 1 para baixar as imagens.

<https://colab.research.google.com/drive/10P97EWEih4XJrb-6gQYYM1IWw60KOLDQ#scrollTo=3CzP-xmHHqh5&uniqifier=1>

Nota (17/08/2024): Este programa não está rodando no computador local com Tensorflow v2.13.1, apesar de rodar corretamente no Google Colab Tensorflow v 2.17.0. Dá erro em *ModelCheckpoint*.

## 2. Transfer learning para classificação M/F

Aparentemente, é difícil aumentar a acuracidade da classificação de rostos em M/F substancialmente acima de 87%, pois o conjunto de dados é pequeno. Num caso como este, é possível fazer “transfer learning” para começar o treino a partir de uma rede com taxa de acerto mais alta, fazer a rede convergir mais rapidamente e atingir uma acuracidade final maior (figura 3). Vamos usar algumas ideias de [<https://machinelearningmastery.com/transfer-learning-for-deep-learning/>] para explicar este conceito.

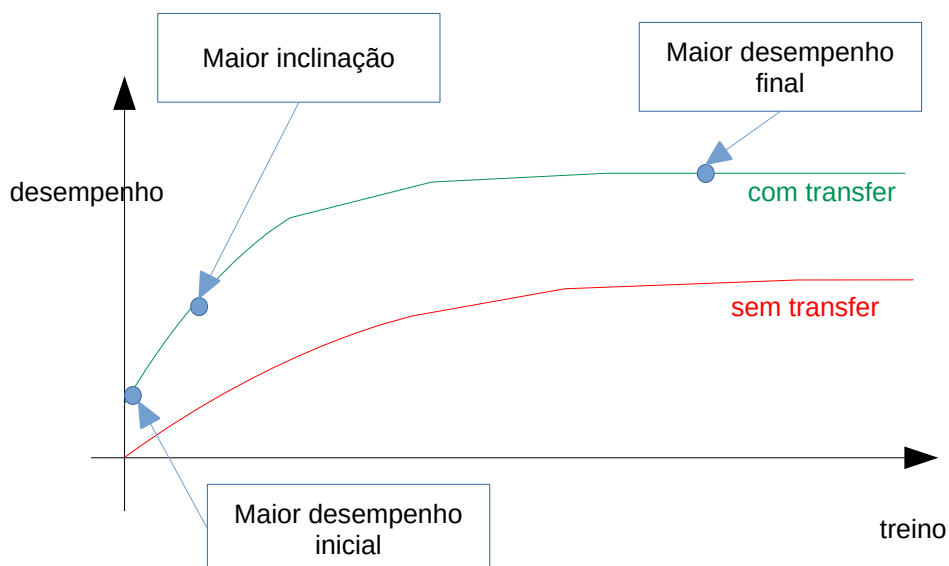


Figura 3: Utilizando “transfer learning”, o desempenho é maior já no início do treino, cresce mais rapidamente e atinge valor final maior do que usar pesos/vieses aleatórios.

*Transfer learning* consiste em reutilizar um modelo pré-treinado como ponto inicial para resolver um novo problema. Isto é, utilizar uma rede neural treinada num outro conjunto de dados (normalmente maior) para resolver um novo problema (normalmente usando um conjunto menor).

Por exemplo, as “redes convolucionais modernas” (VGG, Inception, ResNet, EfficientNet, etc. treinadas em ImageNet 1k) que estudamos na apostila “cifar-ead” podem ser utilizadas como ponto de partida para classificar os rostos em M/F. A explicação intuitiva é que essas redes, para poder classificar as imagens em 1000 categorias, aprenderam a extrair os atributos visuais úteis que também servem para distinguir rosto M/F. Note que as faces humanas não estão incluídas nas 1000 classes de ImageNet:

<https://deeplearning.cms.waikato.ac.nz/user-guide/class-maps/IMAGENET/>

Site [<https://machinelearningmastery.com/how-to-use-transfer-learning-when-developing-convolutional-neural-network-models/>] resume as quatro diferentes técnicas de “transfer learning”:

1. **Classificador:** O modelo pré-treinado é usado para classificar diretamente novas imagens. Por exemplo, digamos que queremos classificar as imagens em “gato”, “cachorro” e “nda”. Para isso, poderia usar qualquer rede treinada em ImageNet que classifica imagens em 1000 categorias. Se a classificação do modelo for alguma das raças de gato (281 “tabby cat” até 285 “Egyptian cat”), a classificação seria “gato”. Se a resposta for alguma das raças de cachorro (151 Chihuahua até 268 Mexican hairless), a resposta seria “cachorro”. Se for alguma classe diferente das anteriores, a classificação seria “nda”.
2. **Extrator de atributos “stand-alone”:** O modelo pré-treinado (sem as camadas de topo) é utilizado para extrair os atributos relevantes da imagem. Estes atributos são usados por outros algoritmos (que não precisam ser necessariamente redes neurais) para fazer a classificação. Poderia usar, por exemplo, árvore de decisão, vizinho mais próximo, boosting, classificador de Bayes, SVM, etc.
3. **Extrator de atributos integrado (ou transfer learning head-only):** O modelo pré-treinado (sem as camadas de topo) é integrado a um novo modelo acrescentando novas camadas (“head”). As camadas do modelo pré-treinado são congeladas durante o treino, isto é, somente as novas camadas são treinadas para resolver o novo problema.
4. **Inicializador de pesos (ou fine tuning):** O modelo pré-treinado (sem as camadas de topo) é integrado ao novo modelo acrescentando camadas “head”, e tanto as camadas do modelo pré-treinado quanto as novas camadas de “head” são treinadas para se adequar ao novo problema. Neste caso, costuma-se utilizar uma *learning rate* bem pequena, para não “estragar” os pesos do modelo pré-treinado.

As técnicas realmente usadas na prática são (3) e (4) e ambas costumam ser usadas em conjunto. Primeiro, utiliza-se transfer learning head-only (técnica 3) com learning rate grande para treinar rapidamente os pesos das novas camadas. Depois, utiliza-se fine tuning (técnica 4) com learning rate bem pequena para fazer calibração fina dos pesos da rede pré-treinada. Vamos utilizar esta ideia usando a rede ResNet50, para classificar rostos em M/F.

**Exercício:** Teste a técnica 2 (extrator de atributos stand-alone) no Cifar-10. Use (por exemplo) ResNet50 e extraia os atributos. Depois, use esses atributos para classificar as imagens em 2 classes com (por exemplo) vizinho mais próximo.

### 3. Transfer learning de ResNet50 para classificação M/F

Vamos fazer transfer learning, usando ResNet50 como modelo-base (backbone), para classificar rostos em M/F (programa 3).

Nota: Antes de executar este programa, deve executar primeiro “célula Colab” do programa 1 para baixar e descompactar o BD localmente.

Estamos redimensionando imagens para  $224 \times 224$ , pois é o padrão de ResNet50 (linhas 41 e 32).

As imagens lidas devem ser tratadas pela função *preprocess\_input* de ResNet50 (linha 35), pois cada modelo possui a sua própria função *preprocess\_input*.

A linha 53 lê a rede ResNet50 com os pesos treinados para ImageNet, sem as camadas do topo da rede (figura 4). A saída desta rede é  $7 \times 7 \times 2048$ . Na saída da rede ResNet50 sem topo, vamos acrescentar as camadas de GlobalAveragePooling, flatten e uma camada densa com 1 neurônio com ativação sigmoide (linhas 55-57).

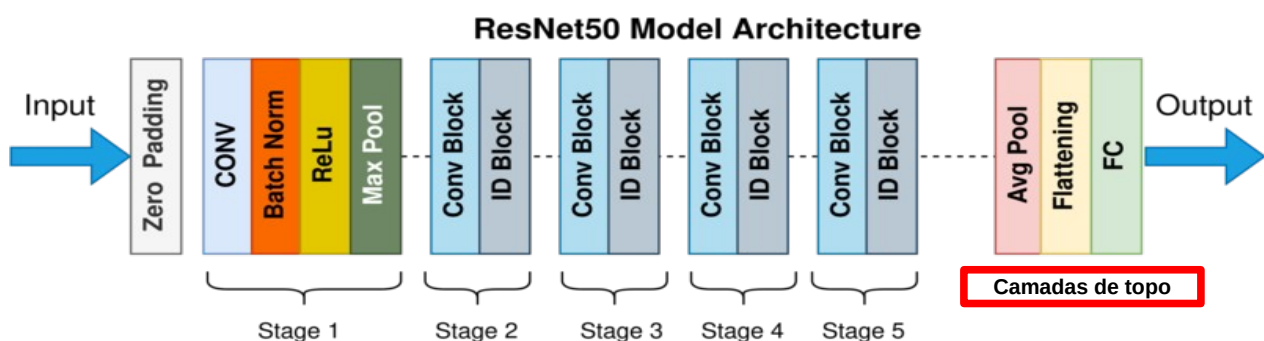


Figura 4: Estrutura de ResNet50 destacando as camadas de topo.

<https://commons.wikimedia.org/wiki/File:ResNet50.png>

As linhas 60-68 treinam somente as duas novas camadas densas do topo durante 40 épocas usando learning rate grande ( $1e-4$ ). As linhas 70-77 treinam todas as camadas durante 40 épocas usando learning rate pequena ( $1e-7$ ).

A saída do programa está na figura 5 e as acuracidades obtidas estão na tabela 2. A acuracidade média de 15 execuções é **96,33%**. Isto é bem mais do que **87,57%** que tínhamos obtido treinando rede “tipo LeNet” do zero. Note na figura 5 que fine tuning melhora a acuracidade inicial dada pelo transfer learning.

Nota: Durante fine tuning, é melhor colocar trainable=False nas camadas de batch normalization. Ajustar os parâmetros de batch normalization pode estragar o funcionamento da rede. Não estou fazendo isso no programa 3.

```
# Fine Tuning
base_model.trainable = True
# Congelar camadas de BatchNormalization especificamente
for layer in base_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False
```





[illegible]

Tabela 2: Acurácias obtidas nas 3 execuções repetidas 5-fold cross validation (15 execuções ao todo) de transfer learning usando ResNet50 como base, usando Tensorflow 2.17 com bug.

**Exercício:** Execute o programa 3 usando Tensorflow sem bug (versão  $\leq 2.15$ ) e verifique a acuracidade obtida. Para não perder tempo, você pode executar uma única repetição de 5-fold cross validation para obter a acuracidade, isto é, coloque na linha 47:

Solução privada em: <https://colab.research.google.com/drive/1FdgHz22rtYINTDE6avG5w30czLjqgbNR>

**Exercício:** Modifique o programa 3 (M/F) para aumentar a taxa de acerto acima de 97%.

Nota: Alunos conseguiram taxas de acerto de mais de 98%.

O que acontece se no programa 3 não carregar os pesos do ImageNet? O programa 4 é igual ao programa 3. Somente troco a linha 53, fazendo com que o programa não carregue os pesos de ImageNet, mas os inicialize com valores aleatórios.

Programa 4: Classifica rostos em M/F, usando ResNet50 com pesos aleatórios. Obtém taxa de acerto muito mais baixa do que fazendo transfer learning.

Executei programa 4 e acuracidade de 73,99% usando Tensorflow 2.17 com bug (figura F). Estes resultados são muito piores do que aqueles obtidos usando transfer learning (96,33% de acerto). São piores até mesmo do que aqueles obtidos usando rede “tipo LeNet” (87,57% de acerto).

```
<<<<<<<<<<<<<<<<<<<<<< Repete indice: 0 <<<<<<<<<<<<<<<<<<<<<<
Acuracidade translearn de fold 0: 52.5000%
Acuracidade fine tuning de fold 0: 70.0000%
Acuracidade translearn de fold 1: 70.0000%
Acuracidade fine tuning de fold 1: 75.0000%
Acuracidade translearn de fold 2: 90.0000%
Acuracidade fine tuning de fold 2: 86.2500%
Acuracidade translearn de fold 3: 40.0000%
Acuracidade fine tuning de fold 3: 68.7500%
Acuracidade translearn de fold 4: 78.7500%
Acuracidade fine tuning de fold 4: 80.0000%
acc_per_fold= [52.49999761581421, 69.9999988079071, 69.9999988079071, 75.0, 89.99999761581421, 86.2500011920929,
40.00000059604645, 68.75, 78.75000238418579, 80.0000011920929]
mean_acc=71.1250%
<<<<<<<<<<<<<<<<<<<<<< Repete indice: 1 <<<<<<<<<<<<<<<<<<<<<<
Acuracidade translearn de fold 0: 72.5000%
Acuracidade fine tuning de fold 0: 72.5000%
Acuracidade translearn de fold 1: 88.7500%
Acuracidade fine tuning de fold 1: 73.7500%
Acuracidade translearn de fold 2: 91.2500%
Acuracidade fine tuning de fold 2: 93.7500%
Acuracidade translearn de fold 3: 33.7500%
Acuracidade fine tuning de fold 3: 52.5000%
Acuracidade translearn de fold 4: 56.2500%
Acuracidade fine tuning de fold 4: 83.7500%
acc_per_fold= [72.50000238418579, 72.50000238418579, 88.7499988079071, 73.7500011920929, 91.25000238418579, 93.75,
33.75000059604645, 52.49999761581421, 56.25, 83.74999761581421]
mean_acc=71.8750%
<<<<<<<<<<<<<<<<<<<<<< Repete indice: 2 <<<<<<<<<<<<<<<<<<<<<<
Acuracidade translearn de fold 0: 75.0000%
Acuracidade fine tuning de fold 0: 72.5000%
Acuracidade translearn de fold 1: 86.2500%
Acuracidade fine tuning de fold 1: 65.0000%
Acuracidade translearn de fold 2: 90.0000%
Acuracidade fine tuning de fold 2: 91.2500%
Acuracidade translearn de fold 3: 60.0000%
Acuracidade fine tuning de fold 3: 58.7500%
Acuracidade translearn de fold 4: 76.2500%
Acuracidade fine tuning de fold 4: 85.0000%
acc_per_fold= [75.0, 72.50000238418579, 86.2500011920929, 64.99999761581421, 89.99999761581421, 91.25000238418579,
60.00000238418579, 58.74999761581421, 76.2499988079071, 85.00000238418579]
mean_acc=76.0000%
<<<<<<<<<<<<<<<<<<<<<<
acc_per_repete= [71.12499982118607, 71.87500029802322, 76.00000023841858]
mean acc=73.0000%
```

Figura F: Acuracidade para classificar rostos em M/F usando ResNet50 inicializada com pesos aleatórios, usando Tensorflow 2.17 com bug.

## II. Classificar Cifar-10 fazendo transfer learning com EfficientNetB0

### 2. Transfer learning para classificar Cifar-10

Adaptado de:

<https://www.kaggle.com/code/nikhilpandey360/transfer-learning-using-xception>

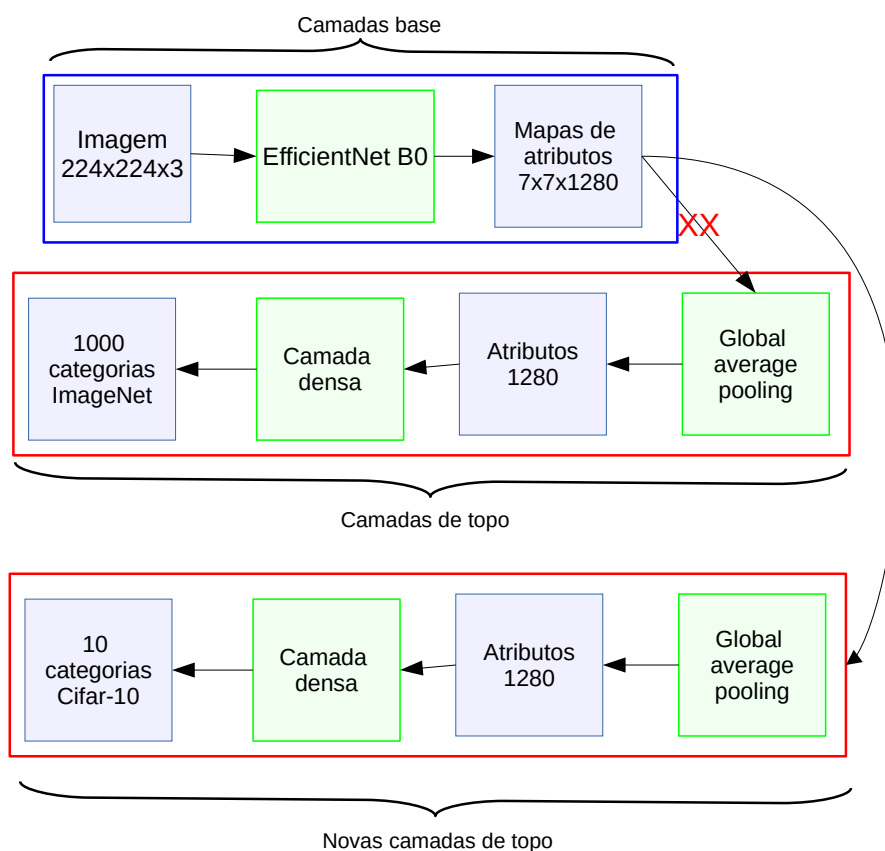
A maior taxa de acerto de Cifar-10 que conseguimos na apostila “cifar-ead” foi de **93,25%** usando ResNet com TTA (test-time augmentation). Provavelmente a taxa de acerto aumentaria um pouco mais usando ensemble, mas não o fizemos pois o processamento demoraria excessivamente. Nesta seção, veremos que usando transfer learning consegue-se taxa de acerto de **96,37%**, sem usar **TTA ou ensemble**. Provavelmente, obteria taxa de acerto ainda maior usando TTA/ensemble.

EfficientNet-B0 possui resolução padrão de entrada  $224 \times 224 \times 3$ . As imagens de Cifar-10 possuem resolução  $32 \times 32 \times 3$ . Assim, as imagens de Cifar-10 serão redimensionadas para  $224 \times 224 \times 3$  para poder fazer transfer learning.

Nota: Testar o que acontece se usar uma resolução mais baixa, por exemplo,  $64 \times 64 \times 3$ .

Vamos fazer data augmentation usando sequência de camadas de pré-processamento:  
`[RandomFlip("horizontal"), RandomRotation(0.1), RandomZoom(0.1)]`

Vamos usar apenas as camadas-base de EfficientNet-B0, eliminando as camadas de topo. Com isso, a rede recebe uma imagem  $224 \times 224 \times 3$  e gera mapas de atributos  $7 \times 7 \times 1280$ . Faremos GlobalAveragePooling desses mapas, gerando um vetor com 1280 atributos. Por fim, colocaremos uma camada densa com 10 neurônios.



Executaremos transfer learning head-only congelando as camadas-base de EfficientNet-B0 durante 20 épocas. Depois, faremos fine tuning, descongelando todas as camadas durante 10 épocas.

Nota: Executando fine tuning por mais épocas melhora ainda mais a acuracidade final.

Nota: Durante fine tuning, é melhor colocar trainable=False nas camadas de batch normalization. Não estou fazendo isso no programa abaixo.

```
# Fine Tuning
base_model.trainable = True
# Congelar camadas de BatchNormalization especificamente
for layer in base_model.layers:
    if isinstance(layer, tf.keras.layers.BatchNormalization):
        layer.trainable = False
```

```

1 #~/deep/algpi/transf/efficientnet_transf_cifar10.py
2 import tensorflow as tf; print("Versao tf:", tf.__version__)
3 import sys; print("Versao python:", sys.version)
4
5 #!pip uninstall tensorflow -y
6 #!pip install tensorflow==2.15
7
8 import tensorflow as tf; print("Versao tf:", tf.__version__)
9 import sys; print("Versao python:", sys.version)
10
11 import matplotlib.pyplot as plt
12 import seaborn as sns
13 import numpy as np; import pandas as pd
14 import warnings; warnings.filterwarnings("ignore")
15 import tensorflow as tf; print(tf.__version__)
16 from tensorflow.keras import Sequential
17 from tensorflow.keras.utils import to_categorical
18 from tensorflow.keras.optimizers import SGD, Adam
19 from tensorflow.keras.callbacks import ReduceLRonPlateau
20 from tensorflow.keras.applications import EfficientNetB0
21 from tensorflow.keras.layers import Input, Flatten, Dense, BatchNormalization, Activation, \
22     Dropout, GlobalAveragePooling2D, MaxPooling2D, RandomFlip, RandomZoom, RandomRotation
23 from tensorflow.keras.datasets import cifar10
24
25 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
26 y_train=to_categorical(y_train); y_test=to_categorical(y_test)
27 print((x_train.shape, y_train.shape)); print((x_test.shape, y_test.shape))
28
29 base_model = EfficientNetB0(include_top=False, weights='imagenet', input_shape=(224,224,3), classes=y_train.shape[1])
30 base_model.trainable = False
31 #base_model.summary()
32
33 data_augmentation = Sequential(
34     [RandomFlip("horizontal"), RandomRotation(0.1), RandomZoom(0.1)]
35 )
36
37 """Consider the image resolution that the imagenet was trained on.
38 The original image resolution of CIFAR-10 is 32x32, which is too low for EfficientNetB0 (min. 71x71)
39 O tamanho padrao de entrada desta rede e' 224x224
40 """
41 inputs = tf.keras.Input(shape=(32, 32, 3))
42 # x = tf.keras.layers.Lambda(lambda image: tf.image.resize(image, (224,224)))(inputs)
43 x = tf.keras.layers.Resizing(224,224)(inputs)
44 x = data_augmentation(x)
45 x = tf.keras.applications.efficientnet.preprocess_input(x)
46 x = base_model(x, training=False) # x = [none,7,7,1280]
47 x = tf.keras.layers.GlobalAveragePooling2D()(x) # x = [none,1280]
48 outputs = tf.keras.layers.Dense(10, activation='softmax')(x)
49 model = tf.keras.Model(inputs, outputs)
50 model.summary()
51
52 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
53 epochs = 20
54 history = model.fit(x_train, y_train, validation_split=0.2, epochs=epochs, verbose=1)
55
56 def plot_history(history):
57     history_frame = pd.DataFrame(history.history)
58     history_frame.loc[:, ['loss', 'val_loss']].plot()
59     history_frame.loc[:, ['accuracy', 'val_accuracy']].plot()
60     return
61 plot_history(history)
62
63 test_loss, test_acc = model.evaluate(x_test, y_test)
64 print("Test accuracy", test_acc); print("Test loss", test_loss)
65
66 """# Fine Tuning"""
67 base_model.trainable = True
68 model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), loss='categorical_crossentropy', metrics=['accuracy'])
69 epochs = 10
70 history = model.fit(x_train, y_train, validation_split=0.2, epochs=epochs, verbose=1)
71
72 test_loss, test_acc = model.evaluate(x_test, y_test)
73 print("Test accuracy", test_acc); print("Test loss", test_loss)
74 plot_history(history)
75
76 """# Confusion Matrix"""
77 class_names=['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
78 predictions=model.predict(x_test)
79 y_pred_classes = np.argmax(predictions, axis=1)
80 y_true = np.argmax(y_test, axis=1)
81 confusion_mtx = tf.math.confusion_matrix(y_true, y_pred_classes)
82 plt.figure(figsize=(12, 9))
83 c = sns.heatmap(confusion_mtx, annot=True, fmt='g')
84 c.set(xticklabels=class_names, yticklabels=class_names)
85
86 #from google.colab import runtime
87 #runtime.unassign()

```

Programa 5: Classifica Cifar-10, fazendo transfer learning de EfficientNet-B0. As linhas 5 e 6 instalam Tensorflow 2.15.

[https://colab.research.google.com/drive/1Q6h1CaoeGJIAk\\_g1d0snrsuxjO-QlyGh](https://colab.research.google.com/drive/1Q6h1CaoeGJIAk_g1d0snrsuxjO-QlyGh)

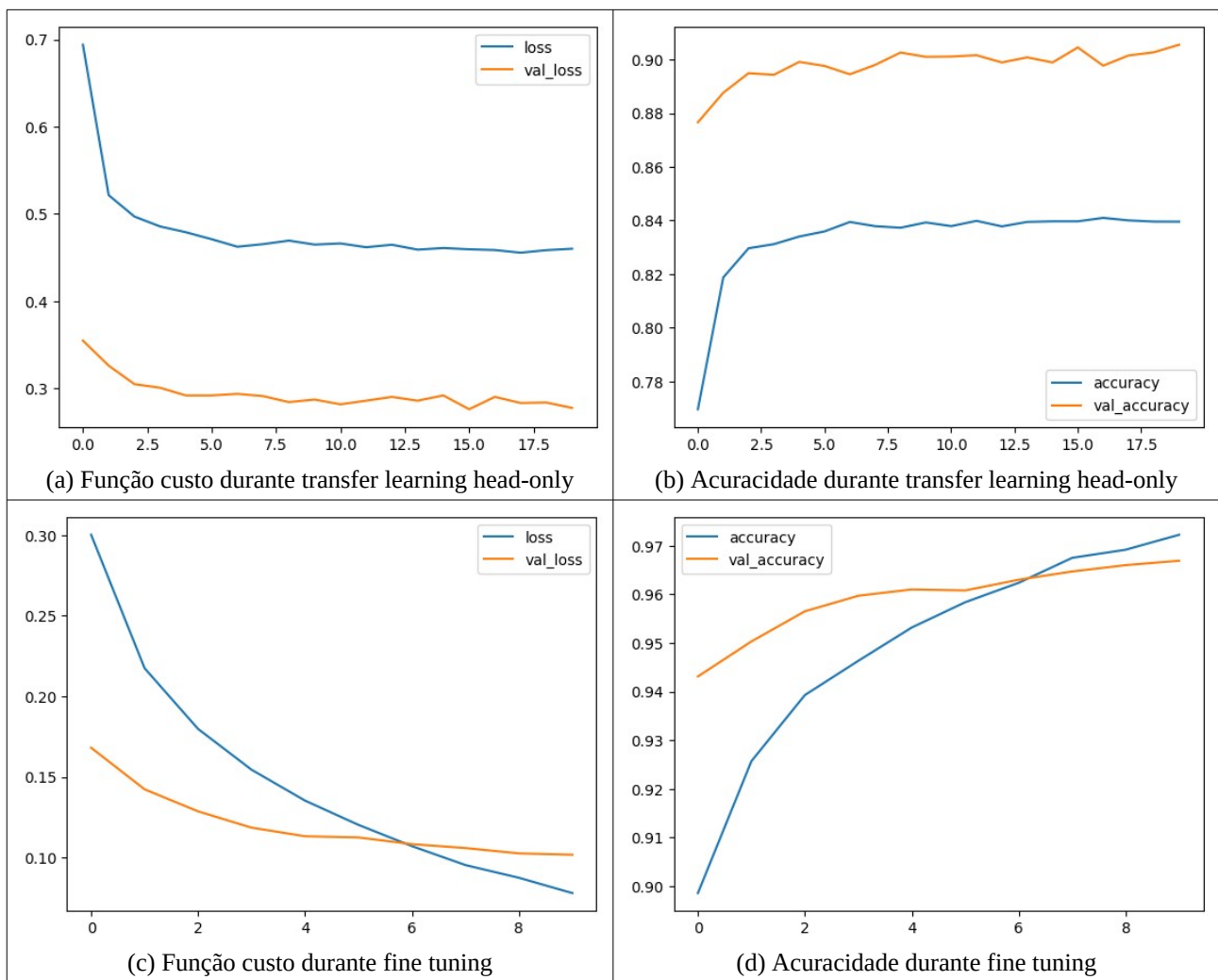


Figura 6: Histórico de treino de transfer learning head-only (a-b) e fine tuning (c-d).



Tabela 4: Acuracidades classificando Cifar-10 fazendo transfer learning do EfficientNetB0, usando Tensorflow 2.15 sem bug.

|                           | Acurac. treino | Acurac. validação | Acurac. teste |
|---------------------------|----------------|-------------------|---------------|
| Após tr. learn. head-only | 83.96%         | 90.55%            | 90.00%        |
| Após fine tuning          | 97.22%         | 96.69%            | 96.37%        |

Atingimos 96,37% de taxa de acerto de teste, a maior taxa de acerto obtida até aqui (que tinha sido 93,4%). Cada época de transfer learning head-only demorou 92s e cada época de fine tuning demorou 300s.

*Nota:* A solução é “meio desonesta”, pois ImageNet possui as mesmas categorias do Cifar-10.

Na matriz de confusão abaixo, podemos observar que muitas vezes gato é confundido com cachorro; automóvel com caminhão; e avião com navio; como era esperado. O trecho do programa que plota matriz de confusão está destacado em amarelo no código.

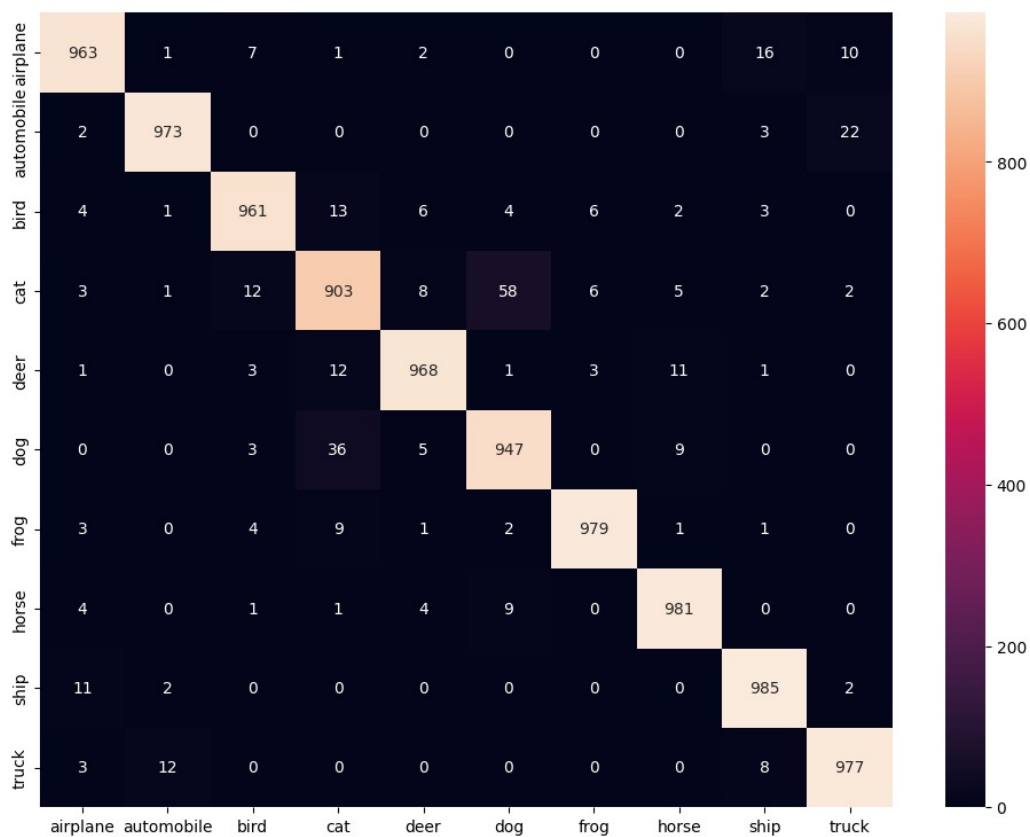


Figura 7: Matriz de confusão com os resultados do programa 5.

*Exercício:* Modifique o programa 5 para classificar fashion-mnist. Para acelerar o processamento, execute somente 10 épocas para fazer “ajuste grosso” e 5 épocas para “ajuste fino”. Qual foi a taxa de acerto fazendo transfer learning? A taxa de acerto foi maior do que aquela obtida sem fazer transfer learning?

*Nota:* Tínhamos obtido 94% *sem* fazer transfer learning.

Solução privada com acuracidade 96,37% em: <https://colab.research.google.com/drive/1Ubj8KP0bZJRPlIDq70i5qUhEbZu7Np4Y#scrollTo=aC4JTeDXiNLH>

*Exercício:* No programa 5, conseguimos atingir taxa de acerto de teste 96,37% ao classificar Cifar-10 fazendo transfer learning de EfficientNet-B0. Melhore o programa 5 e chegar a acuracidade de teste maior que 97% ao classificar Cifar-10.

*Nota:* Não resolvi este exercício. Não sei se é possível resolvê-la.

Possíveis técnicas:

- a) Mudar data augmentation.
- b) Mudar os parâmetros do programa.
- c) Usar regularização L1 ou L2 nas camadas de topo.
- d) Usar função callback para diminuir learning rate quando o desempenho da rede para de melhorar.
- e) Usar learning rate scheduler e/ou CosineDecayRestarts.
- f) Gravar e usar o modelo que atinge a maior taxa de acerto de validação.
- g) Usar TTA (test time augmentation).
- h) Usar ensemble.

*Nota:* Um aluno chegou a 97,72%. Se fizer rotação, zoom, piora. Usar TTA não melhorou. O que melhorou foi fazer deslocamento durante o treino. Aumentou quantidade de épocas.

*Nota:* Um aluno conseguiu chegar a 97% mexendo no learning rate.

## II. Classificar Cifar-10 fazendo transfer learning de ResNet50

Nota (16/08/2024): Como disse acima, estou obtendo resultados do programa 5 substancialmente melhores no computador local com Tensorflow 2.13.1 do que no Colab com Tensorflow 2.17 (veja a tabela T1). Para verificar a hipótese de que este bug está associado ao uso do modelo-base EfficientNetB0, vou mudar o backbone do programa 5 para ResNet50 e verificar se continua havendo diferença entre os resultados do computador local e do Colab.

```
1 #~/haelinux/haepi/deep/algpi/transf/resnet50_transf_cifar10.py
2 import matplotlib.pyplot as plt
3 #import seaborn as sns
4 import numpy as np; import pandas as pd
5 import warnings; warnings.filterwarnings("ignore")
6 import tensorflow as tf; print(tf.__version__)
7 import tensorflow.keras as keras
8 from tensorflow.keras import Sequential, Model
9 from tensorflow.keras.utils import to_categorical
10 from tensorflow.keras.optimizers import SGD, Adam
11 from tensorflow.keras.callbacks import ReduceLROnPlateau
12 from tensorflow.keras.applications.resnet import ResNet50, preprocess_input
13 from tensorflow.keras.layers import Input, Flatten, Dense, BatchNormalization, Activation, \
14     Dropout, GlobalAveragePooling2D, MaxPooling2D, RandomFlip, RandomZoom, RandomRotation
15 from tensorflow.keras.datasets import cifar10
16
17 (x_train, y_train), (x_test, y_test) = cifar10.load_data()
18 y_train=to_categorical(y_train); y_test=to_categorical(y_test)
19 print((x_train.shape, y_train.shape)); print((x_test.shape, y_test.shape))
20
21 base_model = ResNet50(include_top=False, weights='imagenet', input_shape=(224,224,3))
22 base_model.trainable = False
23 #base_model.summary()
24
25 data_augmentation = Sequential(
26     [RandomFlip("horizontal"), RandomRotation(0.1), RandomZoom(0.1)]
27 )
28
29 """Consider the image resolution that the imagenet was trained on.
30 The original image resolution of CIFAR-10 is 32x32, which is too low for EfficientNetB0 (min. 71x71)
31 O tamanho padrao de entrada desta rede e' 224x224
32 """
33 inputs = tf.keras.Input(shape=(32, 32, 3))
34 # x = tf.keras.layers.Lambda(lambda image: tf.image.resize(image, (224,224)))(inputs)
35 x = tf.keras.layers.Resizing(224,224)(inputs)
36 x = data_augmentation(x)
37 x = preprocess_input(x)
38 x = base_model(x, training=False)
39 x = GlobalAveragePooling2D()(x)
40 x = Dropout(0.3)(x)
41 outputs = Dense(10, activation=('softmax'))(x)
42 model = Model(inputs, outputs)
43 model.summary()
44
45 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
46
47 epochs = 20
48 history = model.fit(x_train, y_train, validation_split=0.2, epochs=epochs, verbose=1)
49
50 def plot_history(history):
51     history_frame = pd.DataFrame(history.history)
52     history_frame.loc[:, ['loss', 'val_loss']].plot()
53     history_frame.loc[:, ['accuracy', 'val_accuracy']].plot()
54     return
55 plot_history(history)
56
57 test_loss, test_acc = model.evaluate(x_test, y_test)
58 print("Test accuracy", test_acc); print("Test loss", test_loss)
59
60 """# Fine Tuning"""
61 base_model.trainable = True
62 model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), loss='categorical_crossentropy', metrics=['accuracy'])
63 epochs = 10
64 history = model.fit(x_train, y_train, validation_split=0.2, epochs=epochs, verbose=1)
65
66 test_loss, test_acc = model.evaluate(x_test, y_test)
67 print("Test accuracy", test_acc); print("Test loss", test_loss)
68 plot_history(history)
69
70 """# Confusion Matrix"""
71 class_names=['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
72 predictions=model.predict(x_test)
73 y_pred_classes = np.argmax(predictions, axis=1)
74 y_true = np.argmax(y_test, axis=1)
75 confusion_mtx = tf.math.confusion_matrix(y_true, y_pred_classes)
76 plt.figure(figsize=(12, 9))
77 c = sns.heatmap(confusion_mtx, annot=True, fmt='g')
78 c.set(xticklabels=class_names, yticklabels=class_names)
```

Programa P1: Classifica Cifar-10, fazendo transfer learning de ResNet50.

Tabela T2: Métricas de desempenho para classificar Cifar10 fazendo transfer learning com backbone ResNet50.

|                                        | Computador local<br>Tensorflow v2.13.1 sem bug | Google Colab<br>Tensorflow v2.17.0 com<br>bug | Google Colab<br>Tensorflow v2.15.0 sem bug |
|----------------------------------------|------------------------------------------------|-----------------------------------------------|--------------------------------------------|
| Após 20 épocas de<br>transfer learning | Test accuracy 89.04%<br>Test loss 0.3626       | Test accuracy 88.71%<br>Test loss 0.3729      | Test accuracy 87.79%<br>Test loss 0.4147   |
| Após 10 épocas de<br>fine tuning       | Test accuracy 96.04%<br>Test loss 0.1402       | Test accuracy 95.56%<br>Test loss 0.1489      | Test accuracy 95.43%<br>Test loss 0.1580   |

Os resultados no computador local com Tensorflow 2.13.1, e no Google Colab com Tensorflow 2.17 e 2.15 estão praticamente iguais. O bug parece só acontecer com EfficientNet.

Solução privada em Tensorflow 2.15: <https://colab.research.google.com/drive/1xKS9Ka6WWXe7Vesg7fy5izEIA-tlbGPx1>

Nota: Há Cifar-10 em resolução mais alta obtida fazendo super-resolução. <https://www.kaggle.com/datasets/joaopaulos-chuler/cifar10-128x128-resized-via-cai-super-resolution>

Exercício: Classifique cifar-10 em alta resolução.

**[PSI3471 aulas 13/14, lição de casa extra #3/4. Vale +4]** Copio abaixo o enunciado de EP de PSI3472-2021, adaptando-o para ser feita como um exercício.

<http://www.lps.usp.br/hae/psi3472/ep-2021/index.html>

Solução privada em: /home/hae/haepi/algpi/ep/cat-dog-pos2021

O objetivo deste exercício é identificar se numa imagem aparece gato ou cachorro. Para isso, construa um classificador usando “transfer learning”.

No site <https://www.kaggle.com/rafaelrlima/cat-and-dog-clean-dataset> há um banco de dado com 10.000 imagens, distribuídas em:

Treino: 4.000 imagens de gatos e 4.000 imagens de cachorros.

Teste: 1.000 imagens de gatos e 1.000 imagens de cachorros.

Deixei uma cópia desse BD em:

[www.lps.usp.br/hae/psi3472/ep-2021/cat\\_dog\\_clean.zip](http://www.lps.usp.br/hae/psi3472/ep-2021/cat_dog_clean.zip)

A figura abaixo mostra 10 primeiras imagens “em ordem alfabética” de gatos e cachorros para o treino.

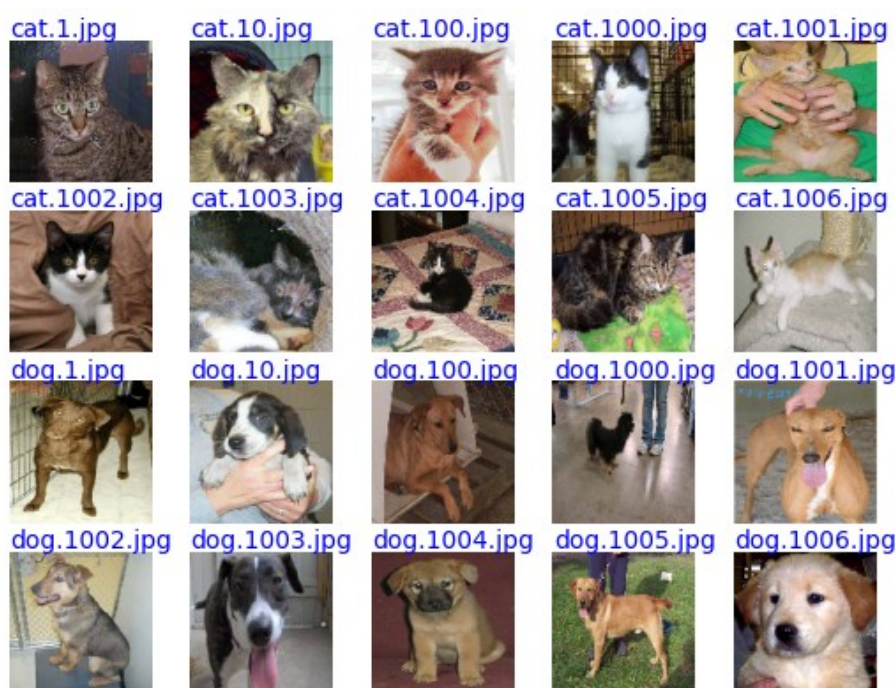


Figura 1: 10 primeiras imagens de treino de gatos e cachorros em ordem alfabética.

Faça dois programas Python3/Keras: treino\_transf.py e teste\_transf.py. O programa treino\_transf.py deve fazer o treino usando “transfer learning”. Você deve obrigatoriamente utilizar algum modelo pré-treinado e adaptá-lo para o problema dado. Você deve gravar o arquivo transf.keras no diretório default (ou no Google Drive) com a rede treinada. Se quiser, pode usar “data augmentation” e outras técnicas para diminuir a taxa de erro. Depois do treino, o programa teste\_transf.py deve carregar transf.keras e classificar as imagens de teste, obtendo a taxa de erro de teste. Também deve imprimir as 10 primeiras imagens de gatos e 10 primeiras imagens de cachorros classificadas incorretamente.

Sem fazer “data augmentation” e sem fazer muito esforço para diminuir erro, obtive taxa de erro de 1,25%. A figura abaixo mostra as 10 primeiras imagens classificadas incorretamente.

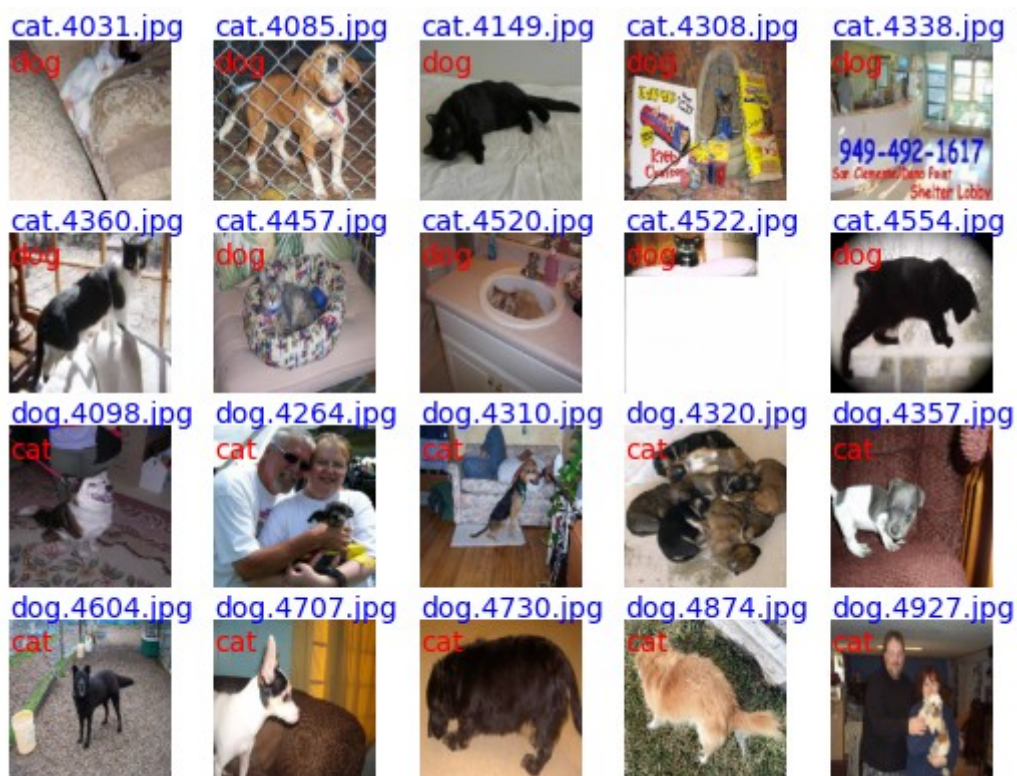


Figura 3: 10 primeiras imagens de teste classificadas incorretamente usando “transfer learning”.

### Ajudas:

1) A célula abaixo baixa e descompacta BD.

```

1 url='http://www.lps.usp.br/hae/psi3472/ep-2021/cat_dog_clean.zip'
2 import os; nomeArq=os.path.split(url)[1]
3 if not os.path.exists(nomeArq):
4     print("Baixando o arquivo",nomeArq,"para diretorio default",os.getcwd())
5     os.system("wget -nc -U 'Firefox/50.0' "+url)
6 else:
7     print("O arquivo",nomeArq,"ja existe no diretorio default",os.getcwd())
8     print("Descompactando arquivos novos de",nomeArq)
9     os.system("unzip -u "+nomeArq)

```

2) O comando abaixo:

```

import glob
cats=glob.glob(nomeDirCat+"/*.jpg"); cats.sort()

```

Cria a lista ordenada “cats” com os nomes de todos os arquivos com extensão “.jpg” do diretório *nomeDirCat*.

3) Se estourar o limite de RAM do Google Colab, você pode:

- Usar *flow\_from\_directory* (veja apostila cifar-ead) ou uma função semelhante que não carrega todo o banco de dados na memória (mas fica carregando do HD quando necessário).
- Carregar imagens com resolução menor do que a resolução padrão das redes pré-treinadas. Por exemplo, para ResNet50 ou EfficientNetB0, usar imagens 112×112 em vez de 224×224 .
- Não usar todas as imagens para fazer treino. Por exemplo, usar 2000+2000 imagens em vez de usar 4000+4000.

**[PSI3471 aula 14. Fim]**



\*) Estou em busca de estudantes dedicados e com excelente histórico acadêmico para orientar em nível de pré-mestrado e mestrado. Caso tenha interesse em desenvolver sua pesquisa comigo, fique à vontade para entrar em contato!

\*) Posteriormente, disponibilizarei uma breve pesquisa de satisfação sobre o curso. O preenchimento levará poucos minutos. Solicito a colaboração de todos nas respostas, pois essas avaliações são essenciais para identificarmos pontos de melhoria nas próximas edições da disciplina.

### [PSI3471] O que veremos na disciplina eletiva do 2º semestre PSI3472

PSI3472 será dada por mim e pelo prof. Márcio. Prof. Márcio vai dar, na segunda parte do curso, sistemas evolucionários e algoritmos genéticos.

Vou dar as técnicas mais avançadas de aplicação de aprendizado de máquina em visão computacional.

**Diferenciação automática:** Vimos que, para treinar uma rede neural, é necessário executar back-propagation que efetua os seguintes cálculos:

$$w_k \rightarrow w'_k = w_k - \eta \frac{\partial C}{\partial w_k}$$

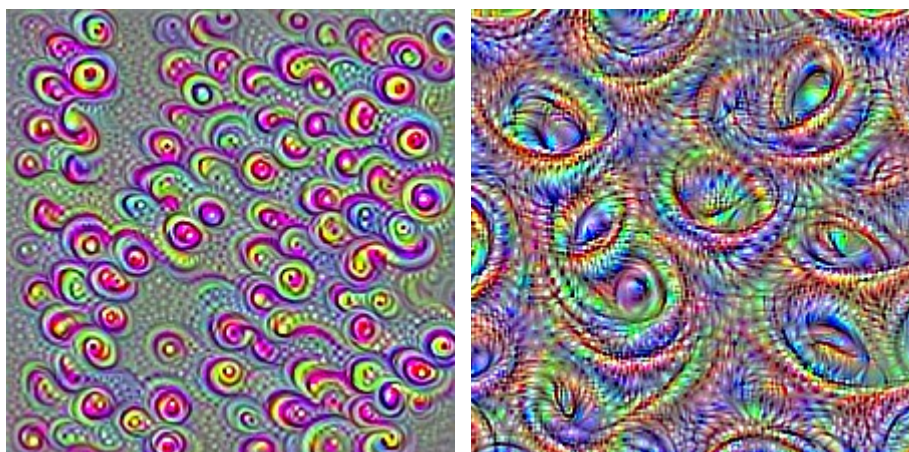
$$b_l \rightarrow b'_l = b_l - \eta \frac{\partial C}{\partial b_l}$$

Como são calculadas as derivadas parciais da função custo  $C$  em relação aos pesos  $w$  e vieses  $b$ ? A rede neural pode ser bem complicado, ter desvios, junções, funções estranhas, etc.

$$\frac{\partial C}{\partial w_k} \text{ e } \frac{\partial C}{\partial b_l}$$

Vamos estudar que isto é feito usando diferenciação automática.

Também veremos que é possível efetuar subida de gradiente (em vez de descida de gradiente) para gerar uma imagem maximiza a saída de um filtro da rede. Aqui, em vez de ajustar os parâmetros, estamos ajustando a imagem de entrada.



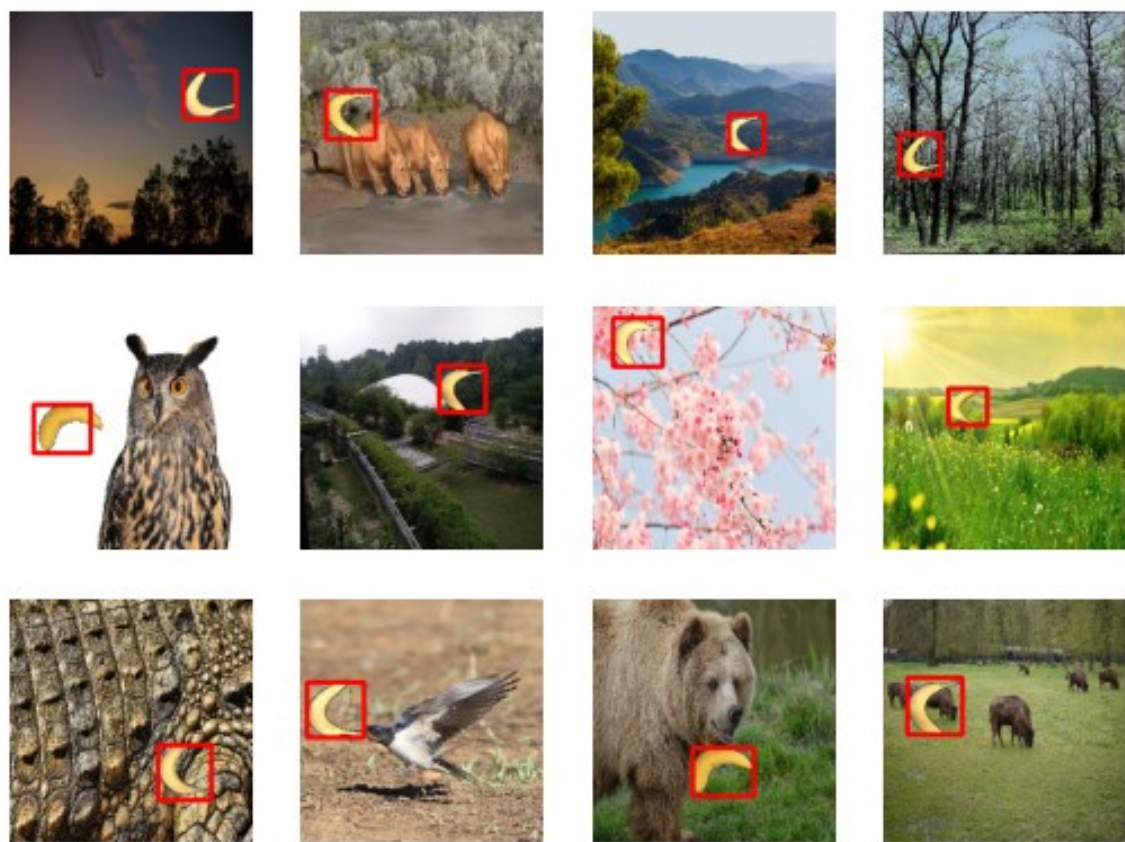
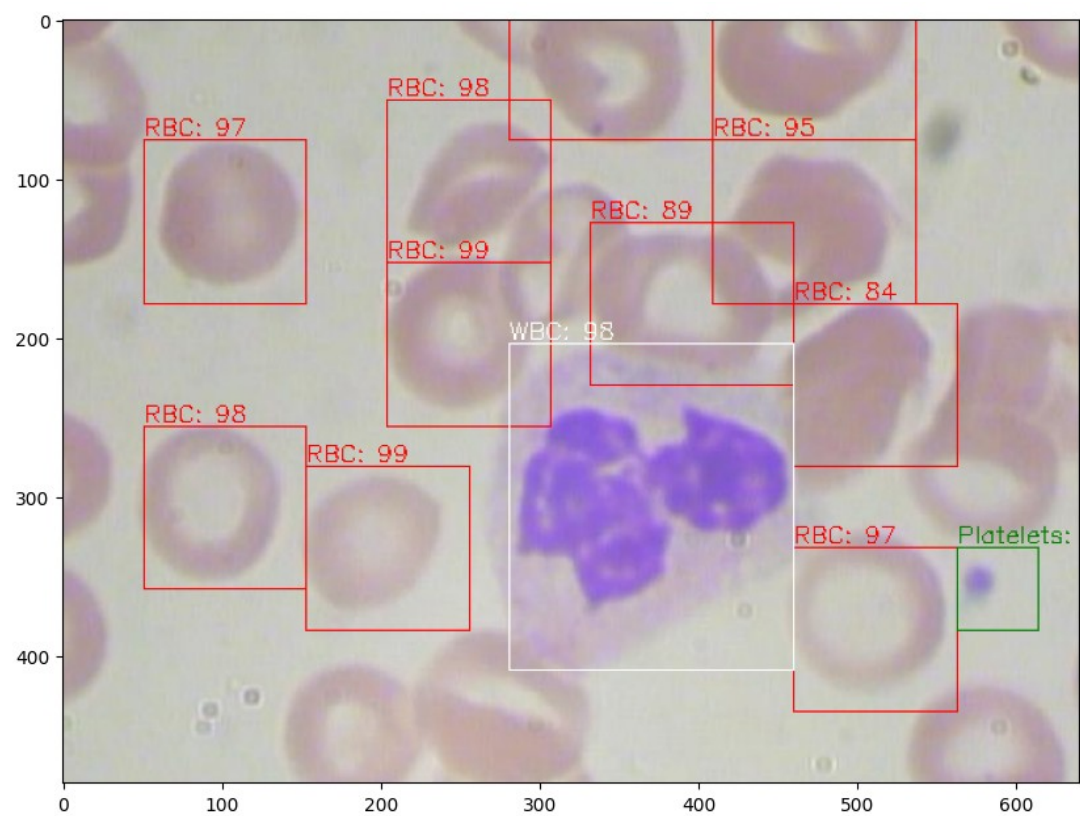
**Vision transformer:** Os modelos de linguagem modernos (LLMs) funcionam baseado em arquitetura chamada Transformer. Essa arquitetura foi testada em problemas de visão com bons resultados quando a quantidade de dados de treino é muito grande. Essa arquitetura chama-se Vision Transformer. Ela pode ser útil em alguns casos, como ter que observar duas regiões distantes na imagem.

**Segmentação:** Estudaremos as técnicas para segmentar uma imagem. Técnicas semelhantes são usadas para colorir imagens em níveis de cinza e para fazer super-resolução. As TVs modernas usam super-resolução para gerar imagem em altíssima resolução (p.ex., 4k) a partir de sinal em resolução menor (p.ex., HD).





**Detecção de objetos:** Estudaremos as técnicas para localizar objetos em imagens.



**Class activation map e grad-cam:** Para que o funcionamento de rede neural não seja “caixa preta”, estudaremos como determinar a região para onde a rede está olhando para decidir que a imagem é de uma certa classe.



CAM da classe “boxer”



CAM da classe “tiger\_cat”

**Few-shot e one-shot learning:** Como conseguimos reconhecer um novo animal a partir de poucas imagens (uma única imagem) de treino?

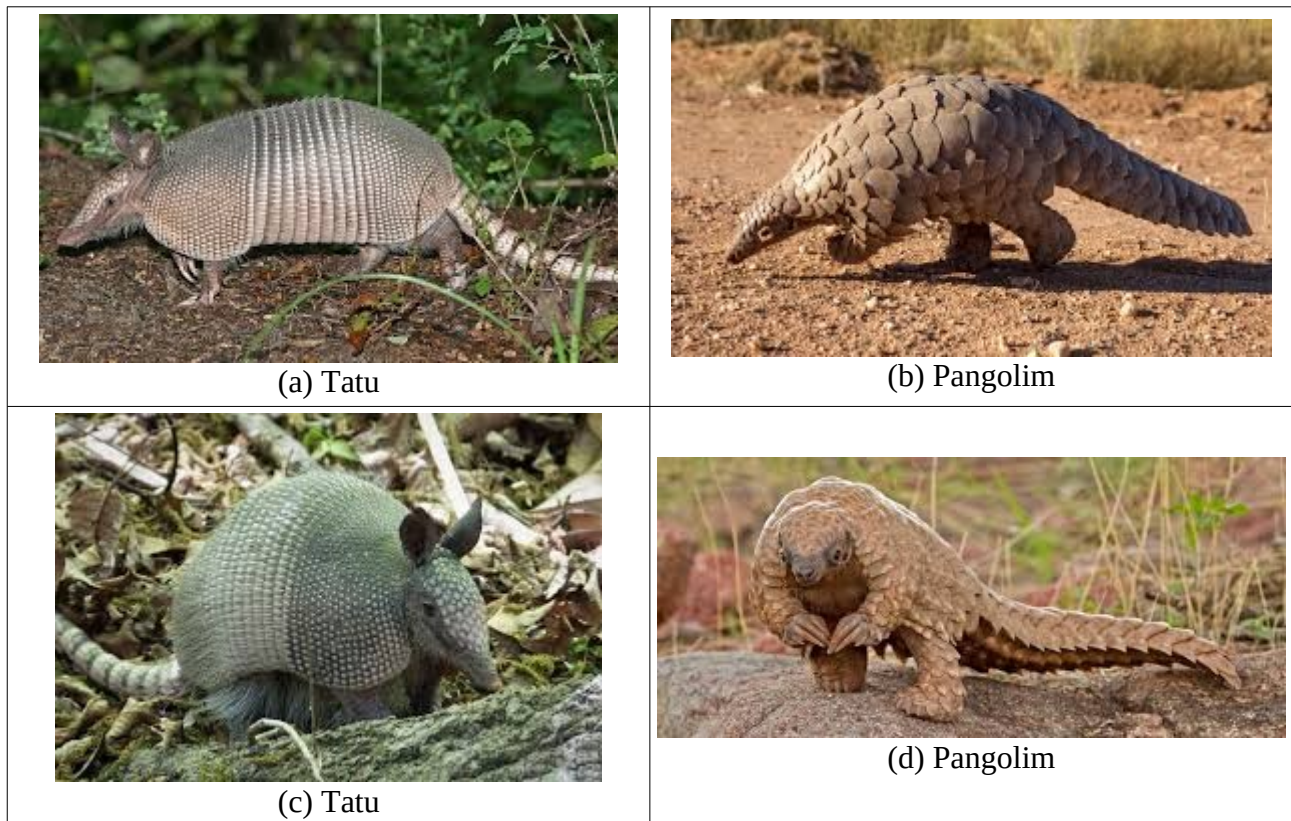


Figura F1: Conjunto suporte, com dois tatus e dois pangolins.

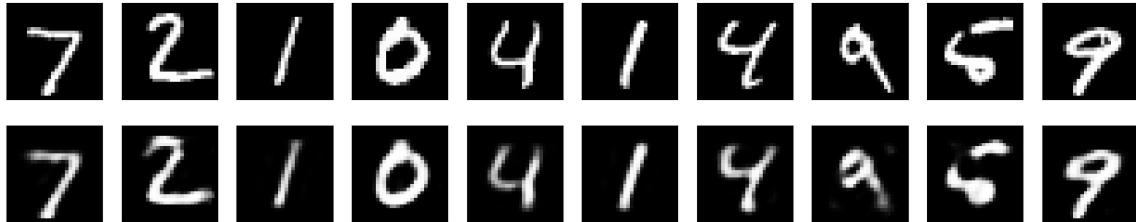


Figura F2: Imagem de busca (query).

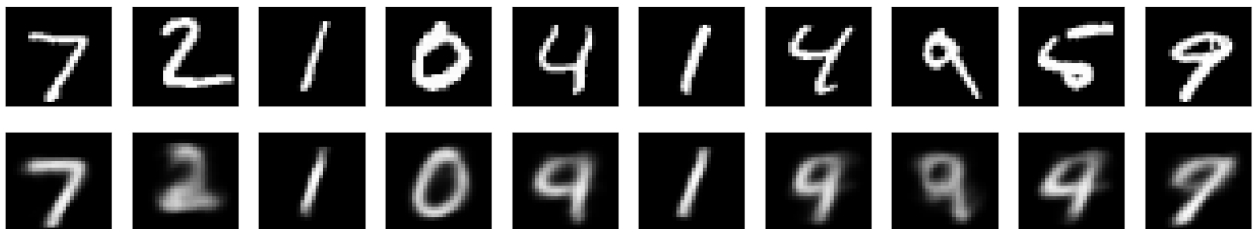
Como funciona reconhecimento facial? Por que não precisa de milhares de imagens de cada morador para poder treinar o sistema de reconhecimento facial do condomínio?

**Auto-encoder:** Auto-encoder pode ser utilizada para diminuir dimensionalidade dos dados. É uma alternativa a técnicas como PCA (Análise de Componentes Principais).

É possível representar as imagens MNIST (com dimensão  $28 \times 28 = 784$ ) usando vetor de dimensão 32?

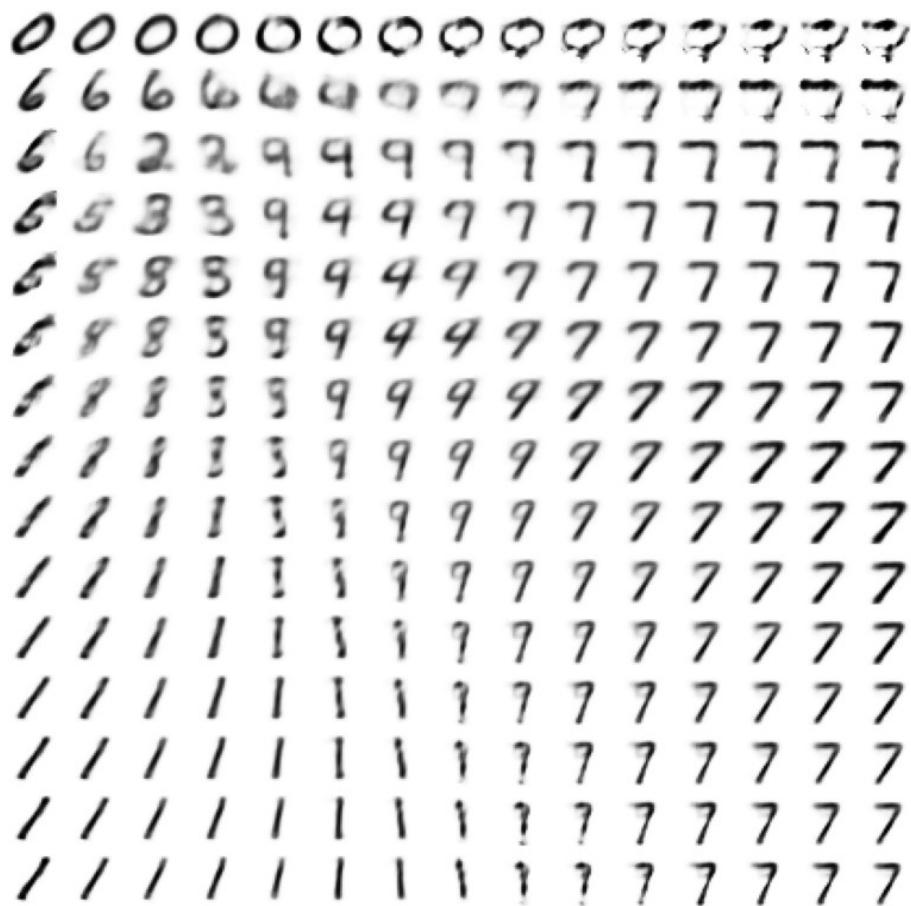
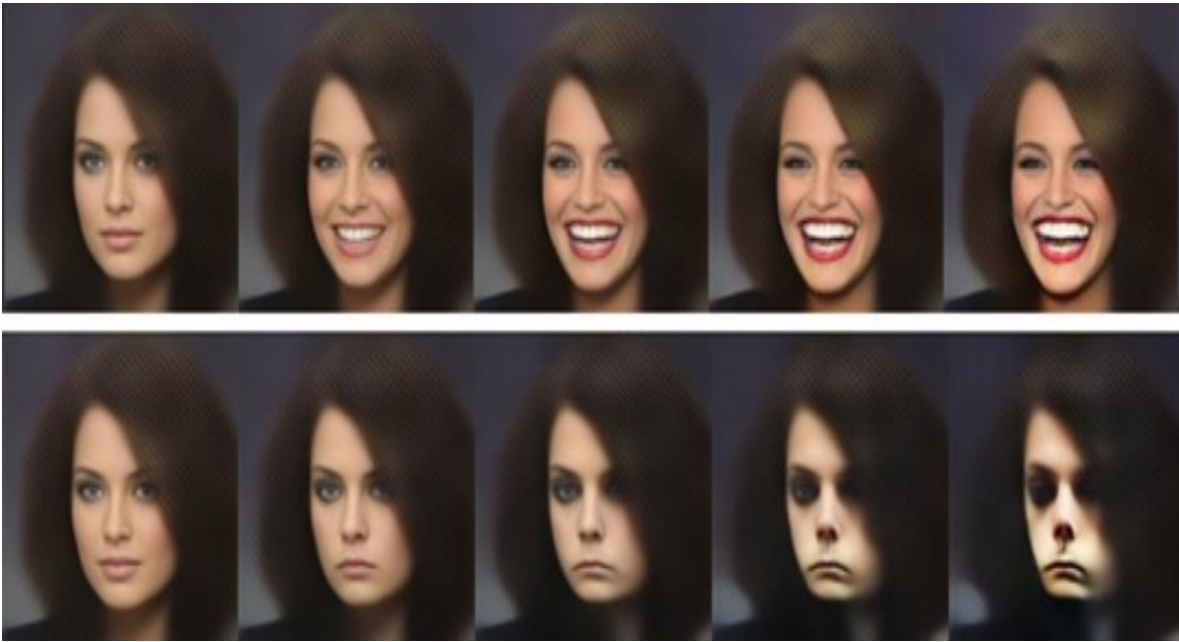


É possível representar as imagens MNIST com 784 elementos usando um vetor de dimensão 2?





É possível que alguma dimensão compactada represente uma certa característica na imagem?



**Generative Adversarial Network (GAN) e modelo de difusão: Como gerar imagens?**

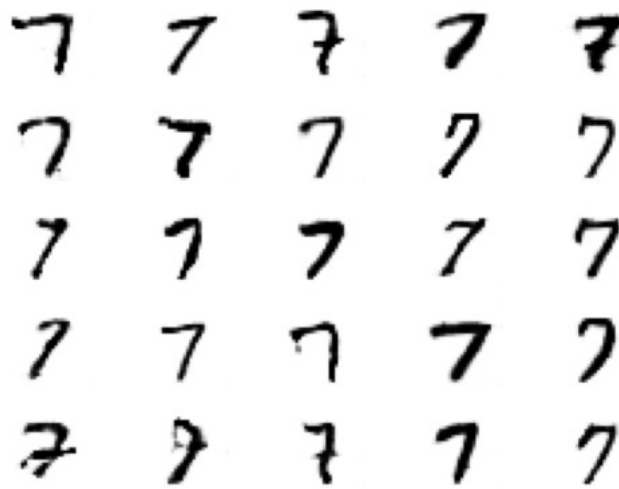


Figura 1: Dígitos “7” gerados artificialmente por uma GAN a partir do MNIST.



Figura 3: Faces artificiais geradas por GAN

# I

SIR WALTER ELLIOT, of Kellynch Hall, in Somersetshire, was a man who, for his own amusement, never took up any book but the Baronetage; there he found occupation for an idle hour, and consolation in a distressed one; there his faculties were roused into admiration and respect, by contemplating the limited remnant of the earliest patents; there any unwelcome sensations, arising from domestic affairs, changed naturally into pity and contempt. As he turned over the almost endless creations of the last century – and there, if every other leaf were powerless, he could read his own history with an interest which never failed – this was the page at which the favourite volume always opened:

‘ELLIOT OF KELLYNCH HALL.

‘Walter Elliot, born March 1, 1760, married, July 15, 1784, Elizabeth, daughter of James Stevenson, Esq. of South Park, in the county of Gloucester; by which lady (who died 1800) he has issue Elizabeth, born June 1, 1785; Anne, born August 9, 1787; a still-born son, Nov. 5, 1789; Mary, born Nov. 20, 1791.’

Precisely such had the paragraph originally stood from the printer's hands; but Sir Walter had improved it by adding, for the information of himself and his family, these words, after the date of Mary's birth – ‘married, Dec. 16, 1810, Charles, son and heir of Charles Musgrove, Esq. of Uppercross, in the county of Somerset’ – and by inserting most accurately the day of the month on which he had lost his wife.

Then followed the history and rise of the ancient and respectable family, in the usual terms: how it had been first settled in Cheshire, how mentioned in Dugdale – serving the office of high sheriff, representing a borough in three successive parliaments, exertions of loyalty, and dignity of baronet, in the first year of Charles II., with all the Marys and Elizabeths they had married; forming altogether two handsome duodecimo pages, and concluding with the arms and motto: ‘Principal seat, Kellynch Hall, in the county of Somerset,’ and Sir Walter's handwriting again in this finale:



Prompt: Faça OCR da imagem anexa.

SIR WALTER ELLIOT, of Kellynch Hall, in Somersetshire, was a man who, for his own amusement, never took up any book but the Baronetage; there he found occupation for an idle hour, and consolation in a distressed one; there his faculties were roused into admiration and respect, by contemplating the limited remnant of the earliest patents; there any unwelcome sensations, arising from domestic affairs, changed naturally into pity and contempt. As he turned over the almost endless creations of the last century – and there, if every other leaf were powerless, he could read his own history with an interest which never failed – this was the page at which the favourite volume always opened:

‘ELLIOT OF KELLYNCH HALL.

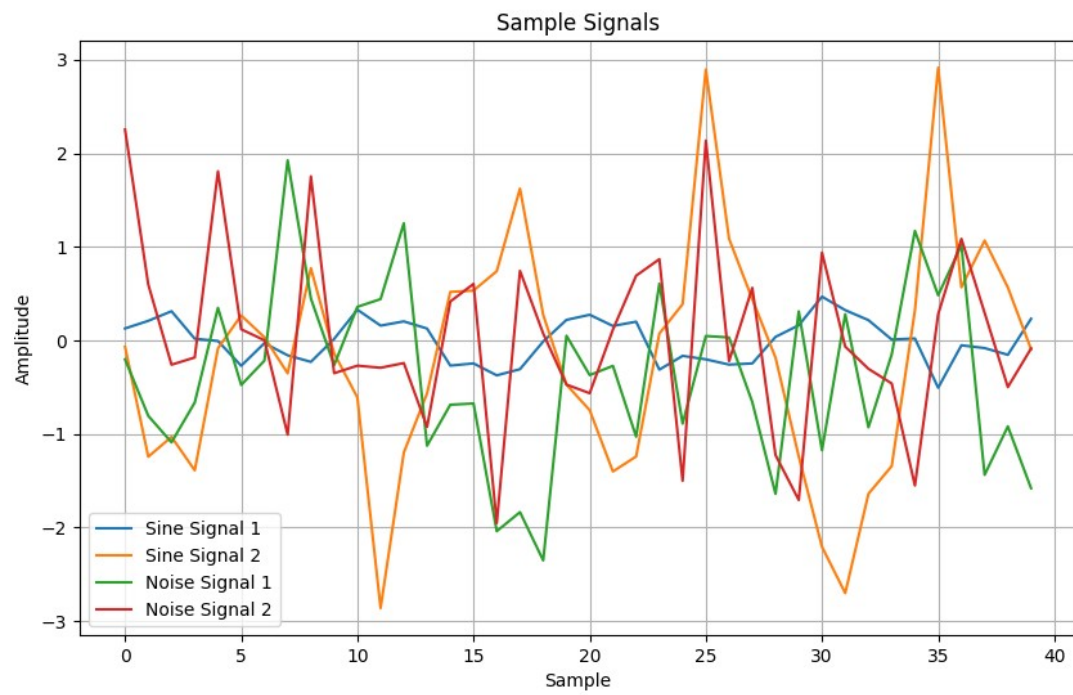
‘Walter Elliot, born March 1, 1760, married, July 15, 1784, Elizabeth, daughter of James Stevenson, Esq. of South Park, in the county of Gloucester; by which lady (who died 1800) he has issue Elizabeth, born June 1, 1785; Anne, born August 9, 1787; a still-born son, Nov. 5, 1789; Mary, born Nov. 20, 1791.’

Precisely such had the paragraph originally stood from the printer’s hands; but Sir Walter had improved it by adding, for the information of himself and his family, these words, after the date of Mary’s birth – ‘married, Dec. 16, 1810, Charles, son and heir of Charles Musgrove, Esq. of Upper-cross, in the county of Somerset’ – and by inserting most accurately the day of the month on which he had lost his wife.

Then followed the history and rise of the ancient and respectable family, in the usual terms: how it had been first settled in Cheshire, how mentioned in Dugdale – serving the office of high sheriff, representing a borough in three successive parliaments, exertions of loyalty, and dignity of baronet, in the first year of Charles II., with all the Marys and Elizabeths they had married; forming altogether two handsome duodecimo pages, and concluding with the arms and motto: ‘Principal seat, Kellynch Hall, in the county of Somerset,’ and Sir Walter’s handwriting again in this finale:

## Classificação de sinais 1D

Distinguir senoide contaminada por ruído de ruído puro.



## Sinais de ECG.

